



Multi-variable Quantification of BDDs in External Memory using Nested Sweeping

Steffan Christ Sølvsten^(✉)  and Jaco van de Pol 

Aarhus University, Aarhus, Denmark
{soelvsten,jaco}@cs.au.dk

Abstract. Previous research on the Adiar BDD package has been successful at designing algorithms capable of handling large Binary Decision Diagrams (BDDs) stored in external memory. To do so, it uses consecutive sweeps through the BDDs to resolve computations. Yet, this approach has kept algorithms for multi-variable quantification, the relational product, and variable reordering out of its scope.

In this work, we address this by introducing the *nested sweeping* framework. Here, multiple concurrent sweeps pass information between each other to compute the result. We have implemented the framework in Adiar and used it to create a new external memory multi-variable quantification algorithm. In practice, this improves Adiar's running time by a factor of 1.7. In turn, this work extends the previous research results on Adiar to also apply to its quantification operation: compared to conventional depth-first implementations, Adiar with nested sweeping is able to solve more problems and/or solve them faster.

Keywords: Time-forward Processing · External Memory Algorithms · Binary Decision Diagrams

1 Introduction

The ability of Binary Decision Diagrams (BDDs) to represent Boolean formulae as small directed acyclic graphs (DAGs) have made them an invaluable tool to solve many complex problems. For example, recently they have been used to check type-and-effect systems [33, 34], to generate proofs for SAT and QBF solvers [13–15], for circuit synthesis [21, 30], to solve games [35, 42, 53], and for symbolic model checking [3, 18, 19, 22, 24, 26, 32].

Implementations of decision diagrams conventionally make use of recursive depth-first algorithms and a unique node table [10, 20, 27, 31, 38, 52]. Both of these introduce random access, which pauses the entire computation while missing data is fetched [28, 37, 41]. For large enough instances, data has to reside on disk and the resulting I/O-operations that ensue become the bottle-neck.

Adiar [49] is a BDD package written in C++ based on the ideas of Lars Arge [5]: the depth-first recursive algorithms are replaced with iterative algorithms. Here, one or more priority queues reorder the execution of recursive calls such that they are synchronised with a level-by-level traversal of the inputs. This makes Adiar’s algorithms, unlike the conventional recursive implementations, optimal in the I/O-model [1] of Aggarwal and Vitter [5, 6]. In turn, this enables it to manipulate BDDs beyond the reach of conventional BDD packages at a negligible cost to its running time [49].

Yet, the ideas in [5, 6, 49] only provide a translation of the simplest BDD algorithms, which do not recurse on the result of other recursive calls. This does not provide a way to translate the more complex BDD algorithms that recurse on intermediate recursion results, e.g. multi-variable quantification. Hence, until this work, Adiar could not easily be used for solving Quantified Boolean formulæ (QBF). Furthermore, game solving and symbolic model checking has until now been out of reach for Adiar.

1.1 Contributions

In Sect. 3, we introduce the notion of *nested sweeping* to provide a framework on which these more complex BDD operations can be implemented. Here, an *outer* bottom-up sweep accumulates the results from multiple nested *inner* sweeps. With this framework in hand, we implement an I/O-efficient multi-variable quantification akin to the one in conventional BDD packages. Furthermore, we identify in Sect. 3.2 optimisations for the nested sweeping framework. The full paper [46] also includes optimisations specific to the quantification operation and an overview of the implementation in Adiar. Section 4 shows that nested sweeping improves the running time in practice by a factor of 1.7 when solving QBF-encodings of two-player games and when reasoning about the transition system in Conway’s Game of Life [23]. We compare our approach to related work in Sect. 5 and provide our conclusions and future work in Sect. 6.

2 Preliminaries

2.1 The I/O-Model

Aggarwal and Vitter introduced the I/O-model [1] to analyse the cost of transferring data to and from a slow storage device. Here, computations can only operate on data that resides in *internal* memory, e.g. the RAM, with a finite size of M . Hence, if the input of size N (or some intermediate result) exceeds M then it needs to be transferred to and from *external* memory, e.g. the disk. Yet, each such data transfer (I/O) moves an entire consecutive block of B elements; an algorithm’s I/O-complexity is the number of I/Os it uses.

One needs $\text{scan}(N) \triangleq N/B$ I/Os to linearly scan through a consecutive list of N elements in external memory [1]. Assuming $N > M$, one needs to use $\Theta(\text{sort}(N))$ I/Os to sort N elements, where $\text{sort}(N) \triangleq N/B \cdot \log_{M/B}(N/B)$ [1]. Furthermore, one can design an I/O-efficient priority queue capable of doing

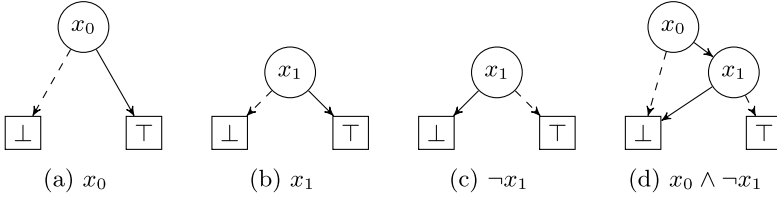


Fig. 1. Examples of Reduced Ordered Binary Decision Diagrams. Terminals are drawn as boxes surrounding their Boolean value. Internal nodes are drawn as circles and contain their decision variable. Arcs to the *high* and *low* child are respectively drawn solid and dashed.

N insertions and deletions in $\Theta(\text{sort}(N))$ I/Os [4]. For simplicity, we overload $\text{scan}(N)$ to be N and $\text{sort}(N)$ to be $N \log_2 N$ when referring to an algorithm's time complexity rather than its I/O complexity.

Intuitively, an algorithm is I/O-inefficient if it uses an entire I/O to retrieve a block but does not make use of a significant portion of the B elements within. That is, random access can result in N I/Os. For all realistic values of N , M , and B , this is several magnitudes larger than both $\text{scan}(N)$ and $\text{sort}(N)$.

2.2 Binary Decision Diagrams

As shown in Fig. 1, a Binary Decision Diagram [12] (BDD) (based on [2, 29]) represents an n -ary Boolean function as a singly-rooted directed acyclic graph (DAG). Each of its two sinks, referred to as *terminals*, contain one of the two Boolean values, $\mathbb{B} = \{\top, \perp\}$. These represent the function's output values. An internal BDD node, v , is associated in $v.\text{var}$ with a Boolean input variable x_i . Furthermore, it has two BDD nodes as children, $v.\text{low}$ and $v.\text{high}$. These three values in f encode the ternary if-then-else $v.\text{var} ? v.\text{high} : v.\text{low}$. What are colloquially referred to as BDDs are in fact *Reduced Ordered* Binary Decision Diagrams (ROBDDs). An Ordered BDD (OBDD) restricts each variable to occur at most once on each path from the root to a terminal and to occur according to a certain order, π . This gives rise to a levelisation of the OBDD where each level, ℓ , is associated with an input variable, x_i . For sake of simplicity, we assume that π is the identity order. A *Reduced* OBDD further restricts the DAG such that (1) no nodes are duplicates of another and (2) no node is redundant, i.e. $v.\text{high} = v.\text{low}$. Assuming the variable ordering, π , is fixed, ROBDDs are a unique canonical form of the Boolean function it represents.

Quantification Algorithm. The levelisation of OBDDs allows the recursive BDD algorithms to both be efficient and elegant. For example, the **or** operation works by a product construction of the two input BDDs. Here, each node of the output BDD simulates, according to π , the decision(s) taken on the shallowest BDD node(s) in the product of nodes from the input.

```

1  exists( $v$ ,  $X$ )
2    if  $v = \perp \vee v = \top$ 
3      return  $v$ 
4     $\text{exi0} \leftarrow \text{exists}(v.\text{low}, X)$ 
5     $\text{exi1} \leftarrow \text{exists}(v.\text{high}, X)$ 
6    if  $v.\text{var} \notin X$ 
7      return Node {  $v.\text{var}$ ,  $\text{exi0}$ ,  $\text{exi1}$  }
8    return or( $\text{exi0}$ ,  $\text{exi1}$ )

```

Fig. 2. A recursive multi-variable **exists** operation.

[$\{(0, 0), \perp, (1, 0)\}; \{(1, 0), \top, \perp\}$]

(a) Node-based representation of $x_0 \wedge \neg x_1$ (Fig. 1d).

[$(0, 0) \rightarrow (1, 0); (0, 0) \rightarrow (1, 1); (1, 0) \rightarrow \perp; (1, 0) \rightarrow \top; (1, 1) \rightarrow \top; (1, 1) \rightarrow \top;]$

(b) Arc-based representation of the **or** of $x_0 \wedge \neg x_1$ (Fig. 1d) and x_1 (Fig. 1b).

Fig. 3. BDD Representations in Adiar.

Since $(\exists x : \phi) \equiv \phi[\top/x] \vee \phi[\perp/x]$, the **or** operation can be used as the basis for an existential quantification ($\exists$) for a set of input variables, $X = \{x_i, x_j, \dots, x_k\}$. As shown in Fig. 2, if v is a terminal then this (sub)BDD depends on none of the to be quantified variables. Otherwise, both its children are resolved recursively into intermediate results, **exi0** and **exi1**. If the decision variable of the root, $v.\text{var}$, should not be quantified, a new node with variable $v.\text{var}$ is created from the two recursive results. Otherwise, **exi0** and **exi1** are instead combined (recursively once more) with a nested **or** operation.

Similarly, one can implement a universal quantification ($\forall$) by use of a nested **and** operation. For clarity, our contributions in Sect. 3 are only phrased with respect to the **exists** operation. But, everything that follows also applies to **forall** by replacing **or** with **and**.

2.3 I/O-Efficient BDD Manipulation

The Adiar [49] BDD package builds on top of Lars Arge’s ideas [5, 6] on how to improve the I/O complexity of BDD manipulation. To not introduce random access, Adiar does not use any hash tables nor recursion for its BDD manipulation. As a result, different BDD objects do not share common subtrees in Adiar. For the same reason, it neither uses pointers to traverse its BDDs. Instead, every BDD node v is uniquely identified by a pair $(v.\text{var}, v.\text{id})$ where $v.\text{id}$ is v ’s index on level $v.\text{var}$. Lexicographically, this *unique identifier* (**uid**) imposes a total ordering of all BDD nodes such that they follow the variable ordering. Yet, the **uid** does not specify the exact index where one can find the BDD node. For example, the BDD for $x_0 \wedge \neg x_1$ in Fig. 1d is represented in Adiar as the list of nodes in Fig. 3a: every node is a 3-tuple with its **uid** followed by the unique identifier of its low and its high children.

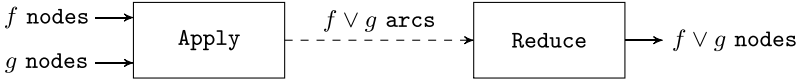


Fig. 4. The Apply-Reduce pipeline of **or** in Adiar.

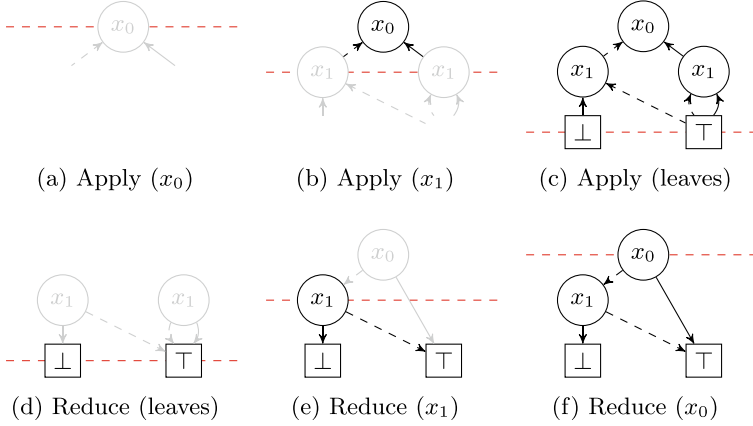


Fig. 5. Step-by-step example of the **or** of x_1 (Fig. 1b) and $x_0 \wedge \neg x_1$ (Fig. 1d) with time-forward processing; subfigures show the state after processing each level. Arcs in gray are pushed to the algorithm’s priority queue whereas the ones in black have been written to the output file.

As depicted in Figs. 4 and 5, the previous BDD operations in Adiar, such as **or**, process a BDD with two sweeps. Both sweeps use *time-forward processing* [4, 17] to achieve their I/O-efficiency: computation is deferred with one or more priority queues until all relevant data has been read. During the first sweep, the *Apply*¹, the entire recursion tree is unfolded top-down. Here, the priority queues also double as a computation cache [10, 38] by merging separate paths to the same recursion target. Hence, the resulting output is in fact not a tree but a DAG. Yet, it is only an OBDD and needs to be reduced. To do so, Adiar uses an I/O-efficient variant of the original bottom-up Reduce algorithm by Bryant [4, 12]. Here, a priority queue is used to forward the *uid* of reduced nodes t' in the final ROBDD to their to be reduced parents s in the intermediate OBDD. Yet, to know the parents s , the Reduce needs the intermediate OBDD to be transposed, i.e. the DAG’s edges to be reversed. Luckily, the Apply sweep outputs its arcs (directed edges) sorted by their target². This effectively transposes the OBDD and so no extra work is needed for the Reduce [5, 49]. For example, the **or** of Figs. 1b and

¹ Similar to [43], we refer to all top-down manipulating sweeps as Apply, e.g. **not**, **or**, and **if-then-else**. This even includes **identity** which merely reverses the edges.

² In [49], arcs to terminals are actually output to a separate file sorted by their source. This is merely to improve performance. Hence, we will ignore this detail.

1d creates the unreduced BDD in Fig. 5c with the arc-based representation in Fig. 3b.

The I/O and time complexity of this Apply–Reduce tandem is $\mathcal{O}(\text{sort}(N + T))$, where N is the size of the input(s) and T is the size of the unreduced output of the Apply sweep [49]. To catch up with conventional implementation’s performance, major efforts have been dedicated to improve on this foundation.

Levelised Cuts [51]. The arcs placed in the above-mentioned priority queues correspond to graph cuts in the (R)OBDDs. In particular, these cuts correspond to the border between the processed and to be processed nodes. Since the priority queues induce a levelised processing order, the shape of these cuts match the (R)OBDD’s levels.

This ensures that the maximum size of the priority queues is bounded by the maximum levelised cut in the input. Hence, sound upper bounds on these cuts can in turn be used to determine a priori whether a faster internal-memory priority queue can be used.

In practice, this improves performance for smaller and moderate instances.

Levelised Random Access [50]. Orthogonally, an Apply sweep for a product construction, e.g. an **or**, can be simplified if one of its inputs is narrow, i.e. each level fits into internal memory. In this case, one can load each level in its entirety into internal memory. This provides random access to all of its nodes on said level, making one of two priority queues within the Apply (as in [49]) obsolete.

In practice, this improves performance for larger instances.

3 I/O-Efficient Multi-variable Quantification

Our previous work in [49] only covers simple BDD operations without any data-dependencies in its recursion, e.g. the **or**. Yet, this does not cover the **exists** in Fig. 2, where the nested call to **or** on line 8 depends on the recursions from lines 4 and 5.

To address this, we introduce the *nested sweeping* framework. As shown in Figs. 6 and 7, nested sweeping wraps the algorithm(s) depicted in Fig. 4: after transposing the input in an initial Apply sweep, a single *outer* Reduce sweep

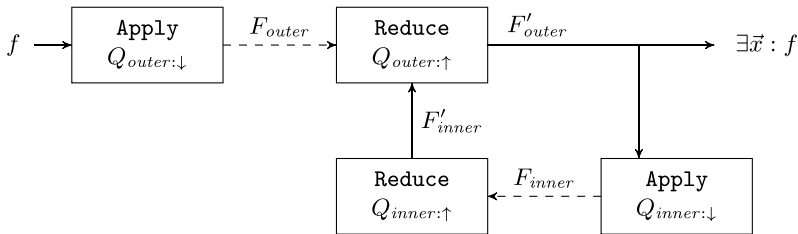


Fig. 6. The Apply–Reduce pipeline of **exists** with Nested Sweeping. F_* are files created by the respective sweep while $Q_{*:*}$ are the priority queues they each use.

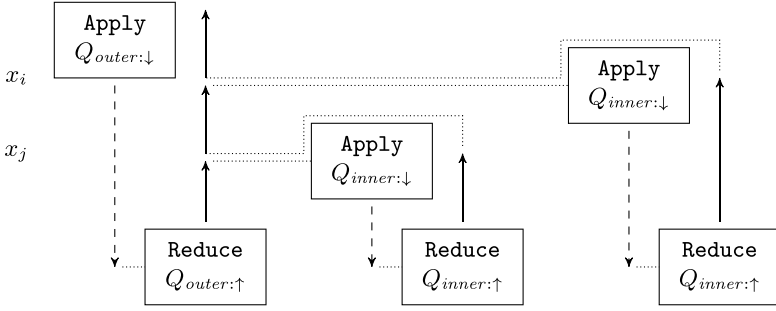


Fig. 7. Sweep direction (solid/dashed) and control-flow (dotted) of Nested Sweeping. The y-axis corresponds to the levels within the BDD. Variables x_i and x_j induce nested sweeps; for **exists**, they are to be quantified variables.

accumulates the result of multiple *inner* Apply–Reduce sweeps. For **exists**, the inner Apply is an **or** sweep.

More precisely, nested sweeping consists of the four phases described below and depicted step-by-step in Fig. 9.

Outer Apply: As shown in Fig. 8, inputs are combined (and possibly manipulated) in an Apply sweep into a single file, F_{outer} . This transposes and merges the inputs such that they are of the form needed by the Reduce of [49].

In the case of **exists**, this is merely a simple transposition of f . In the full paper [46], we explore ways in which this phase can also do double duty for partially resolving the quantification computation.

Outer Reduce: As in [49], each level of F_{outer} is reduced bottom-up by having a priority queue, $Q_{outer:↑}$, forward the information about reduced nodes, t' , to their unreduced parents, s . The reduced output is pushed into a new file, F'_{outer} .

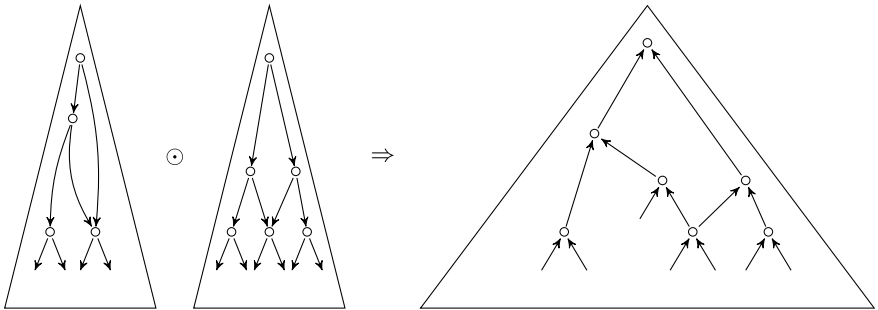


Fig. 8. Outer Apply one (or more) input BDD(s) are processed in a top-down sweep to create a single transposed BDD.

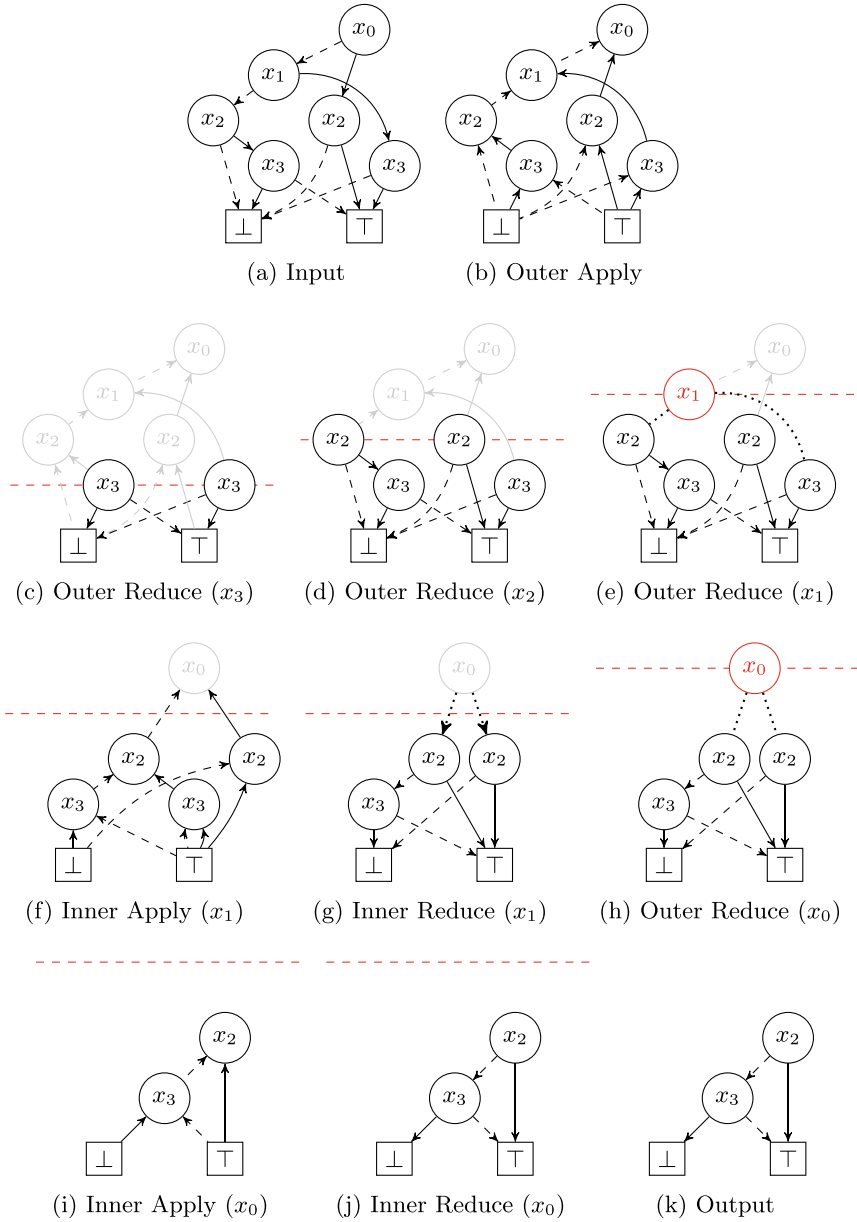


Fig. 9. Step-by-step example of existential quantification of x_0 and x_1 with nested sweeping; each subfigure shows the state after each respective step. The grey nodes depict unreduced nodes created in 9b while dashed red lines show the latest resolved level in the outer Reduce. The red nodes and dotted lines in 9e and 9h depict Case 3 of the outer Reduce. The dotted arcs in 9g depict Case 3 of the inner Reduce. (Color figure online)

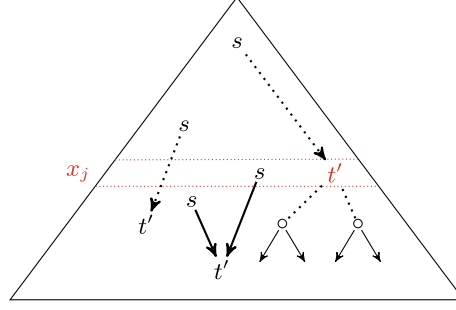


Fig. 10. Outer Reduce: solid arcs stay in $Q_{outer:\uparrow}$ (Case 2a) while dotted arcs are turned into requests for $Q_{inner:\downarrow}$ (Cases 2b on the left and 1 on the right).

Let x_j be the next level that needs a nested sweep. For **exists**, x_j is the largest still to be quantified variable in X . As visualised in Fig. 10, the logic of [49] is extended as follows:

1. If the current level is x_j , each arc $s \rightarrow t'$ to a reduced node t' at this level is turned into a request and placed in a second priority queue, $Q_{inner:\downarrow}$. For **exists**, the requests are of the form $s \rightarrow (t'.low, t'.high)$.
2. If the current level is deeper than x_j , nodes are reduced as in [49] with a caveat: whether the arc $s \rightarrow t'$ to the reduced node t' is placed in $Q_{outer:\uparrow}$ or in $Q_{inner:\downarrow}$ depends on the level of the unreduced parent s as follows:
 - (a) If $x_j \leq s.var$, i.e. s is as deep or deeper than level x_j , then $s \rightarrow t'$ is placed in $Q_{outer:\uparrow}$ as normal.
 - (b) Otherwise, i.e. if $s.var < x_j$, $s \rightarrow t'$ is placed in $Q_{inner:\downarrow}$ instead.

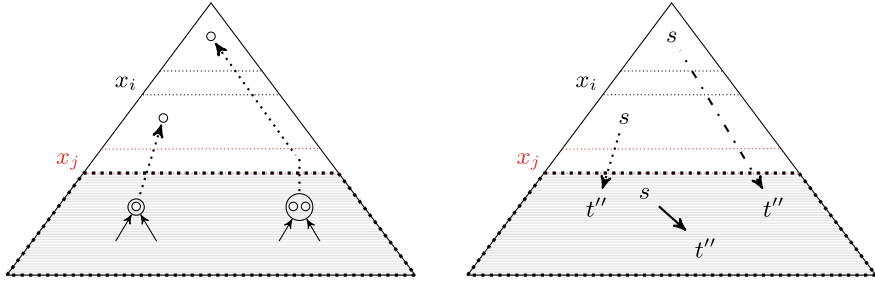
For **exists**, Case 1 matches the invocation of **or** on line 8 of Fig. 2 whereas 3 is the return with an unquantified variable on line 7.

When level x_j has finished processing, $Q_{inner:\downarrow}$ is populated with all requests that span across level x_j . Now, the inner Apply sweep is invoked.

Inner Apply: As depicted in Fig. 11a, starting with the requests in $Q_{inner:\downarrow}$, the reduced nodes, t' , placed in F'_{outer} by the outer Reduce sweep, are processed with an Apply sweep from [49]. The intermediate unreduced result is placed in a new file, F_{inner} .

For **exists**, this sweep is the execution of the **or** on line 8 of Fig. 2. While the algorithms in [49] were only applied to a single BDD, they can also be applied to an entire forest; this merely requires prepopulating their priority queue with recursion requests to each root. Hence, we can reuse the previous top-down algorithms from [49] as is.

Inner Reduce: After the inner Apply sweep, F_{inner} is reduced in another bottom-up Reduce sweep of [49]. This creates the reduced nodes t'' placed in a new file



(a) Inner Apply: starting with multiple root requests (dotted), F_{inner} is constructed with the to-be preserved subtrees (left) together with new nodes that are products of previous ones (right).

(b) Inner Reduce: arcs below x_j stay within the inner sweep (Case 1, solid). Arcs that cross x_j are given back to the outer (Case 2, dotted) or to the next inner sweep (Case 3, dash dotted).

Fig. 11. Visualization of the Inner Apply and the Inner Reduce.

F'_{inner} . Let x_i be the next level above x_j that also needs a nested sweep. For **exists**, x_i is the largest variable smaller than x_j that also needs to be quantified. The arc $s \rightarrow t''$ is placed in a priority queue, $Q_{inner:\uparrow}$, as follows.

1. If $x_j < s.\text{var}$, i.e. the parent s is below level x_j , then $s \rightarrow t''$ is forwarded within this inner Reduce sweep's priority queue, $Q_{inner:\uparrow}$.
2. If $s.\text{var} \in [x_i, x_j]$, i.e. the parent s is between level x_i and x_j then $s \rightarrow t''$ is given back to the outer sweep, $Q_{outer:\uparrow}$. This matches Case 2a in the outer Reduce.
3. If $s.\text{var} < x_i$, i.e. the parent s is above x_i , then $s \rightarrow t''$ is placed into $Q_{inner:\downarrow}$ to prepare the next invocation of an inner Apply sweep. This matches Case 2b in the outer Reduce.

The three cases above are depicted in Fig. 11b. For **exists**, Cases 2 and 3 are equivalent to the return from **or** back to **exists**. Case 1 is equivalent to a return statement within the **or**'s own recursion. Case 3 is needed to match 2b with x_j replaced with x_i .

Finally, F'_{inner} replaces F'_{outer} and control returns to the outer Reduce sweep to proceed with the levels above x_j .

As shown in Fig. 9, F_{inner} from the inner Apply sweep can be thought of as overlayed on top of F_{outer} from the outer Apply sweep; together they produce a valid (but unreduced) OBDD. The outer and inner Reduce sweeps work together to reduce this into a single file, F'_{outer} . When no more levels, x_j , need to be processed and the outer Reduce sweep has finished processing, then F'_{outer} contains the final reduced BDD of all nested operations.

As mentioned above, the inner Apply sweep needs its priority queue $Q_{inner:\downarrow}$ to be prepopulated with all relevant roots. This is done in Cases 1 and 2b in the outer and Case 3 in the inner Reduce sweeps.

Since the result of the inner Apply and Reduce sweeps replaces the entire set of nodes in F'_{outer} , the priority queue $Q_{inner;\downarrow}$ not only needs to be populated with requests for the nodes that need to be changed but also with requests for the nodes one wishes to keep (see also Figs. 9e and 11a). This makes the inner Apply sweep not only compute the desired result but also act as a mark-and-sweep garbage collection. On the first glance, these additional non-modifying requests may seem too costly – especially if most requests do not modify subtrees. In practice, 33.3% of all requests created throughout our benchmarks in Sect. 4 are subtree modifying. For each benchmark instance, 23.0% of all requests modify subtrees on average (median 35.6%). That is, a reasonable number of all requests (and hence BDD nodes processed) change the subgraph in F'_{outer} .

3.1 Complexity of Nested Sweeping

As mentioned in the description of the outer Apply sweep, nested sweeping works for multiple inputs. In this work, it suffices to assume it only has to deal with a single BDD f of N nodes as also depicted in Figs. 6 and 9.

Lemma 1. *A single BDD f with N nodes can be transposed in $\Theta(\text{sort}(N))$ I/Os and time and $\Theta(N)$ space.*

Proof. In $\Theta(\text{scan}(N))$ I/Os and time iterate over and split all nodes v in-order into the two arcs $v.\text{uid} \rightarrow v.\text{low}$ and $v.\text{uid} \rightarrow v.\text{high}$. Sort these $2N$ arcs on their target using $\Theta(\text{sort}(N))$ I/Os and time and linear space to transpose them.

Lemma 2. *Ignoring the work done within the inner sweeps, the outer Reduce sweep costs $\Theta(\text{sort}(N))$ I/Os and time and requires $\mathcal{O}(N)$ space.*

Proof. This follows from the complexity of the Reduce algorithm in [49] (based on [6]) and the constant extra time spent for each of the N' nodes to resolve the additional logic in Cases 1 and 2 of the outer Reduce sweep.

By combining Lemmas 1 and 2 together with the fact that the last invocation of the inner Apply and Reduce sweeps constructs, together with the outer Reduce sweep, the output of size T , we obtain the following lower bound on the complexity of nested sweeping.

Corollary 1. *Nested Sweeping uses $\Omega(N + T)$ space and $\Omega(\text{sort}(N + T))$ time and I/Os where N and T are the size of the input and output, respectively.*

In particular for the **exists** BDD operation, let T_j be the size of F'_{outer} when the inner Apply sweep is invoked at level x_j .

Lemma 3. *A single invocation of the inner Apply and Reduce sweeps at x_j costs $\Theta(\text{sort}(N' + T_j^2))$ I/Os and time and uses $\mathcal{O}(N' + T_j^2)$ space.*

Proof. As in [49], the algorithm's complexity depends on the number of elements placed in the priority queues [4]. In particular, a single nested **or** sweep deals with up to $2N$ arcs from F_{outer} . On top of these, it also processes up to $2T_j^2$ arcs created during the product construction of F'_{outer} .

Since nested sweeping closely simulates the (parallelised) recursive BDD algorithm in Fig. 2, one should expect it achieves, similar to the algorithms in [49], major improvements in the number of I/Os at the cost of a log-factor in the running time when compared to the conventional recursive algorithms. This is indeed the case.

Proposition 1. *Quantification of a set of variables, X , is computable with nested sweeping in $O(\text{sort}(N^{2^{|X|}}))$ I/Os and time and $O(N^{2^{|X|}})$ space.*

Proof. Due to Lemma 1, F_{outer} from the outer Apply sweep has up to $2N$ arcs. This is also the size of F'_{outer} without any inner sweeps. Each inner Apply and Reduce sweep may increase the size of F'_{outer} quadratically. The result follows from Lemmas 3 to 2.

Asymptotically, this is not an improvement over just quantifying each variable one-by-one using the algorithm already proposed in the full version of [49]. Yet, doing so would involve $2|X|$ sweeps over *all* levels of the input whereas, as highlighted in Figs. 7 and 9, nested sweeping only processes levels below each quantified variable. This difference is also evident in practice: throughout our benchmarks in Sect. 4, when quantifying with nested sweeping rather than each variable independently, the total number of requests processed with the **or** operation decreases by 13.9% while the share of 2-ary product constructions increases from 57.3% to 66.6%.

3.2 Optimisations for Nested Sweeping

While nested sweeping as described above is an improvement over previous work in [49], there are multiple avenues to further improve its performance in practice.

Bail-Out of Inner Sweep: There is no need for the outer Reduce sweep to invoke the inner sweeps if $Q_{\text{inner}:\downarrow}$ only contains requests that preserve subtrees, i.e. if Case 1 in the outer Reduce did not create any requests that manipulate the accumulated OBDD in F'_{outer} . On level x_j , such requests can stem from a redundant node t' being suppressed. For **exists**, this may also occur due to either $t'.\text{low}$ or $t'.\text{high}$ being the $\perp$ terminal, which is neutral for the **or** operation, or being $\top$, which is short-circuiting it.

In this case, the entire content of $Q_{\text{inner}:\downarrow}$ can be redistributed between $Q_{\text{outer}:\uparrow}$ and $Q_{\text{inner}:\downarrow}$ for the next deepest to be quantified level, x_i . After doing so, the outer Reduce sweep can immediately proceed processing the next level.

For **exists**, any short-circuiting by the **or** operation in Case 1 of the outer Reduce sweep can kill off some subtrees in F'_{outer} . In this case, one cannot skip the last invocation of the inner sweeps. Otherwise, the final result F'_{outer} could include dead nodes. Yet, even so, one can instead of the expensive top-down algorithm, e.g. **or** for **exists**, invoke the inner Apply sweep with a much simpler (and therefore faster) mark-and-sweep algorithm.

In practice, 75.6% of all nested sweeps in our benchmarks (see Sect. 4 for their presentation) are skippable. For each benchmark, between 6.8% and 93.5% of all nested computations were skipped with an average of 59.2% (median of 81.0%). The number of nested levels depends on the problem domain and its instance. For the Garden of Eden (GoE) benchmark, only 26.8% of all nested computations were skipped on average (median 29.0%), whereas 82.7% (median 84.3%) of all levels of the Quantified Boolean Formulas (QBFs) could be skipped.

Root Requests Sorter: Instead of making the outer Reduce sweep push requests directly into $Q_{inner:\downarrow}$, it can push it into an intermediate list of requests, $L_{outer:\downarrow}$. The content of $L_{outer:\downarrow}$ is sorted using the same ordering as $Q_{inner:\downarrow}$ as the inner Apply sweep is invoked, to then merge it on the fly with the inner Apply sweep’s priority queue. This allows one to postpone initialising this priority queue until the inner Apply sweep is invoked. This has multiple benefits:

- $Q_{inner:\downarrow}$, resp. $Q_{inner:\uparrow}$, only exists and uses internal memory during the inner Apply sweep, resp. the inner Reduce sweep. Hence, the memory otherwise dedicated to $Q_{inner:\downarrow}$ can be used in the outer Reduce sweep for the Reduce’s per-level data structures in [49]. Furthermore, this also increases the amount of space available to the inner Reduce sweep. Hence, this ought to improve the running time of both the inner and the outer Reduce sweeps.
- If $Q_{inner:\downarrow}$, resp. $Q_{inner:\uparrow}$, is initialised for each inner Apply sweep, resp. inner Reduce sweep, then the monotonic and faster *levelised priority queue* in the full version of [49] can be used instead of a regular non-monotonic priority queue.
- Levelised cuts [51] bound the size of the priority queues in each individual inner Apply and Reduce sweep. Hence, for each nested sweep, one can, if it is safe to do so, replace $Q_{inner:\downarrow}$ and/or $Q_{inner:\uparrow}$ with a faster internal memory variant.
- Levelised random access [50] changes the order in which nodes are resolved on each level. This has to be accommodated for in the sorting predicate in $Q_{inner:\downarrow}$. Hence, $L_{outer:\downarrow}$ allows for this optimisation to be applied for each invocation of the inner Apply sweep depending on the width of F'_{outer} .

Furthermore, levelised cuts not only bound the size of $Q_{outer:\uparrow}$ in the outer Reduce sweep but also the size of $L_{outer:\downarrow}$. Hence, while deciding whether $Q_{outer:\uparrow}$ fits into memory, one can also decide whether $L_{outer:\downarrow}$ does.

All in all, this allows the optimisations in [50, 51] to be applied on a sweep-by-sweep basis. In practice, if one neither uses faster internal-memory variants of $Q_{inner:\downarrow}$ and $Q_{inner:\uparrow}$ nor levelised random access, then Adiar needs a total of 32.1 h to solve 145 out of the 147 benchmarks in Sect. 4. Using these two optimisations shaves 13.0 h off the total computation time (speedup of 1.68). For each individual instance, this improves Adiar’s performance between a factor of 1.07 and 5.05 (1.77 on average). Furthermore, without $L_{outer:\downarrow}$, the exponential blow-up in Proposition 1 implies $Q_{inner:\downarrow}$ would almost always have to use external memory. As the optimisations in [49–51] would then not be applicable, one would expect a slowdown of several orders of magnitude similar to [51].

4 Experimental Evaluation

We have implemented the nested sweeping framework in Sect. 3 in Adiar v2.0. To evaluate the impact of this addition, we have run experiments aiming at answering the following three research questions:

1. How does nested sweeping compare to the repeated use of the single-variable quantification from the full version of [49]?
2. How does Adiar with nested sweeping compare to the external memory BDD package, CAL [43]?
3. How does Adiar with nested sweeping compare to conventional BDD packages that use depth-first recursion and memoisation [9, 20, 25, 31, 52]?

As we are only concerned with the design of BDD algorithms, not when and how to use them, we will not compare to non-BDD based approaches.

4.1 Benchmarks

We have implemented the following two benchmarks that rely on multi-variable quantification. Similar to [48, 49], all benchmarks have been implemented on top of C++-templated adapters for each BDD package. This makes each BDD package run the exact same set of operations without introducing any indirection. The source code for all benchmarks can be found at the following url:

github.com/ssoelvesten/bdd-benchmark

QBF Solving: Given a Quantified Boolean Formula (QBF) in the QCIR [54] format, each gate of the given circuit is recursively transformed into a BDD. For inputs, we use the 102 encodings from [44] of 2-player games on a grid. In our experience, the symbolic style of these inputs makes them well suited to be solved with BDDs. Hence, they provide a typical use-case of quantification in BDDs. Furthermore, though these inputs are not in CNF they are in prenex form. In practice, resolving these prenex quantifications at the end is computationally much more expensive than computing the to be quantified circuit, i.e. the *matrix*.

Based on preliminary experiments, we use a variable order based on a depth-first traversal of the given circuit. In the prenex, we merge adjacent blocks with the same quantifier to increase the number of concurrently quantified variables.

Garden-of-Eden: A *Garden-of-Eden* [39] (GoE) is any configuration of a cellular automaton without a predecessor. For Conway’s Game of Life on a grid of size $n_r \times n_c$, this can be encoded as a relation with $(n_r + 2) \cdot (n_c + 2)$ *previous-state* variables, $\vec{x}$, and $n_r \cdot n_c$ *next-state* variables, $\vec{x}'$. Next-state variables, $x'_i \in \vec{x}'$, in the BDD follow a row-major order. Previous-state variables, $x_i \in \vec{x}$, are interleaved to directly precede their respective next-state variable, x'_i . A GoE corresponds to an unsatisfying assignment to $x'_i \in \vec{x}'$ after having quantified all previous-state variables, $\vec{x}$.

Recent results show there exists no GoE for Conway’s Game of Life of size 8×8 or smaller [8]. Hence, the BDD for the $n_r \times n_c \leq 8 \times 8$ sized transition relation will collapse to $\top$ after existential quantification. Yet, the row-major encoding of the transition relation itself only requires a polynomially sized BDD. Hence, the complexity of this problem manifests during the existential quantification.

4.2 Hardware and Settings

As in [48–51], we have run our experiments on the *Grendel* cluster at the Centre for Scientific Computing Aarhus. In particular, we ran both benchmarks on machines with 48-core 3.0 GHz Intel Xeon Gold 6248R processors, 384 GiB of RAM, 3.5 TiB of SSD disk, and run Rocky Linux (Kernel 4.18.0–513). All code was compiled with GCC 10.1.0 or `rustc` 1.72.1. Each BDD package was given $\frac{9}{10}$ th of the available RAM, i.e. 345 GiB, leaving $\frac{1}{10}$ th to other data structures and the operating system. Next to that, the BDD packages use a single thread and their default/recommended settings.

Note that these machines have vast amounts of memory. This is to ensure that depth-first implementations are not slowed down by external factors. If less memory is available, then depth-first implementations would have to run multiple garbage collections to stay within the memory limits (cf. the largest instances solved by BuDDy [31] in Fig. 14a). This, in turn, clears their memoisation tables and forces them to recompute previous results. Furthermore, this large amount of memory ensures they can solve larger problems without using the swap partition. If they had to use it then they would slow down by about two orders of magnitude (see the full paper of [49] for an example). Hence, machines of this scale allow us to measure the algorithms’ running time without the noise otherwise introduced by their execution environment. Finally, this biases the running time in favour of the depth-first implementations, which in turn makes the numbers we report on Adiar’s relative performance close to the worst-case.

Adiar does not yet support dynamic variable reordering. Hence, we have disabled reordering for all other BDD packages as part of these experiments.

4.3 Experimental Results

The computing cluster’s scheduler does not let many long-running jobs run concurrently. To obtain all 1176 data points reported below within only a few months, we had to place each of the 147 instances in buckets of instances with a common timeout. In particular, an instance is placed in the bucket with the smallest timeout that is four times larger than Adiar needed during preliminary experiments. That is, a BDD package timing out should only be understood as it (possibly) being considerably slower than Adiar.

Depending on an instance bucket placement, running time measurements were made 1 to 3 times. Due to node failures on the cluster, Adiar with nested sweeping was run once more, resulting in its measurements being repeated on many instances 4 times. On average, all data points had 3.0 measurements.

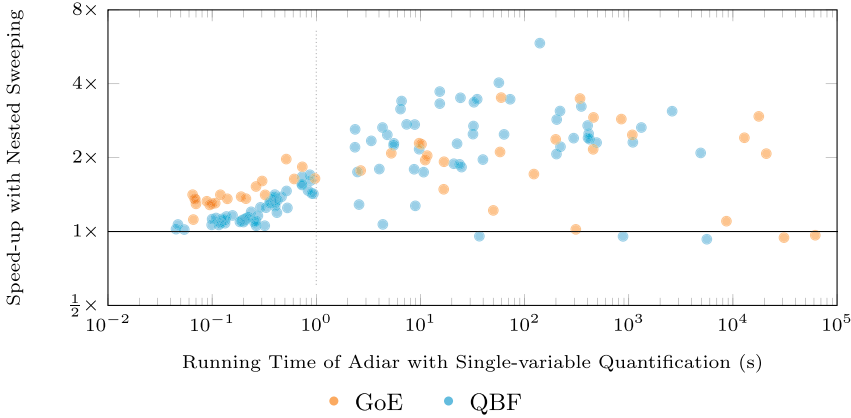


Fig. 12. Relative time ($T_{\text{old}}/T_{\text{new}}$) of quantification with nested sweeping (T_{new}) compared to the previous repeated single-variable quantification (T_{old}).

Similar to [48, 49, 51], we report for each benchmark the minimum time recorded as it is the measurement with the least noise [16]. Average ratios are aggregated using the geometric mean.

Adiar needs less than 1 s to solve 27 out of the 45 GoE instances, resp. 50 out of the 102 QBF instances. As will become evident later with Figs. 13 and 14, this makes them so small that they are not within the current scope of Adiar. For completeness, we still show and discuss these results.

RQ 1: Improvement by Nested Sweeping. Figure 12 shows the speed-up of using Adiar with nested sweeping relative to quantifying each variable individually. Across all instance sizes, nested sweeping is in general an improvement in performance. We have recorded a slowdown of up a factor of 1.05 for 5 instances. Yet, we also recorded speed-ups up to a factor of 5.88 for the 142 remaining instances. On average, performance improves by a factor of 1.7 for both QBF and GoE. The total computation time was decreased by 21% from 49.4 h to 39.1 h.

RQ 2: Comparison to CAL. To the best of our knowledge, CAL [43] (based on [7, 40]) is the only other BDD package also designed to manipulate BDDs larger than main memory. To do so, it uses breadth-first algorithms that should work well with BDDs stored on disk via the operating system’s swap memory [43]. CAL also includes algorithms to support multi-variable quantification [43]. For more details, see [43] and Sect. 5. The machines for our experiments provide a 48 GiB swap partition, i.e. a 12.5% increase in available space.

Preliminary experiments indicated CAL’s breadth-first algorithms are much slower than Adiar’s time-forward processing. Hence, we multiplied the timeout for CAL by a factor of 3. But as is evident in Fig. 13, this increase turned out to still overestimate CAL’s performance on larger instances. Hence, the running

Table 1. Time taken and the average ratio between Adiar and CAL for the 124 commonly solved instances. The average (geometric mean) pertains only to instances where Adiar needed at least 1 s to solve them. Ratios larger than 1.00 means Adiar is faster.

	Time		# Solved		Avg. Ratio (1+s)	
	GoE	QBF	GoE	QBF	GoE	QBF
Adiar	7431.9 s	688.0 s	45	102	—	—
CAL	184688.3 s	295660.0 s	38	86	5.0	25.2

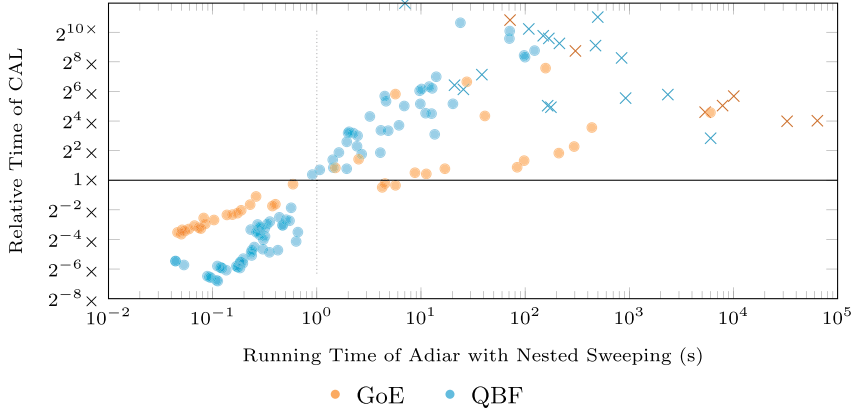


Fig. 13. Relative performance ($T_{\text{CAL}}/T_{\text{Adiar}}$) of CAL (T_{CAL}) compared to Adiar with Nested Sweeping (T_{Adiar}). Time-/Memouts are marked as crosses.

times and averages in Fig. 13 and Table 1 pertain only to the 124 instances which CAL can solve within the given RAM, SWAP, and the time limits.

Even though this discards the instances where CAL struggles, i.e. the data points that remain are in CAL’s favour, Fig. 13 shows Adiar heavily outperforms CAL for instances where Adiar takes 1 s or longer to solve. Where CAL uses 133.4 h to solve 124 instances, Adiar, by solving them in only 2.3 h, is 59.1 times faster. On these larger instances, CAL is on average 14.7 times slower than Adiar. As is evident in Fig. 13 and Table 1, Adiar especially outperforms CAL on the QBF benchmark. For example, the largest difference was measured for the `hex/hein_15_5x5-13` QBF instance, where CAL is 1081 times slower than the 71.2 s Adiar needs to solve it.

CAL is considerably faster for the instances where Adiar takes less than 1 s to solve. At this small scale, CAL uses the conventional depth-first algorithms [51] to sidestep the performance issues of its external memory breadth-first algorithms. Doing the same for Adiar is still left as future work [50, 51].

RQ 3: Comparison To Depth-First Implementations. We have compared the performance of Adiar with BuDDy 2.4 [31], CUDD 3.0.0 [52], LibBDD 0.5 [9], OxiDD 0.6 [25], and Sylvan 1.8.1 [20]. Their individual performance relative to

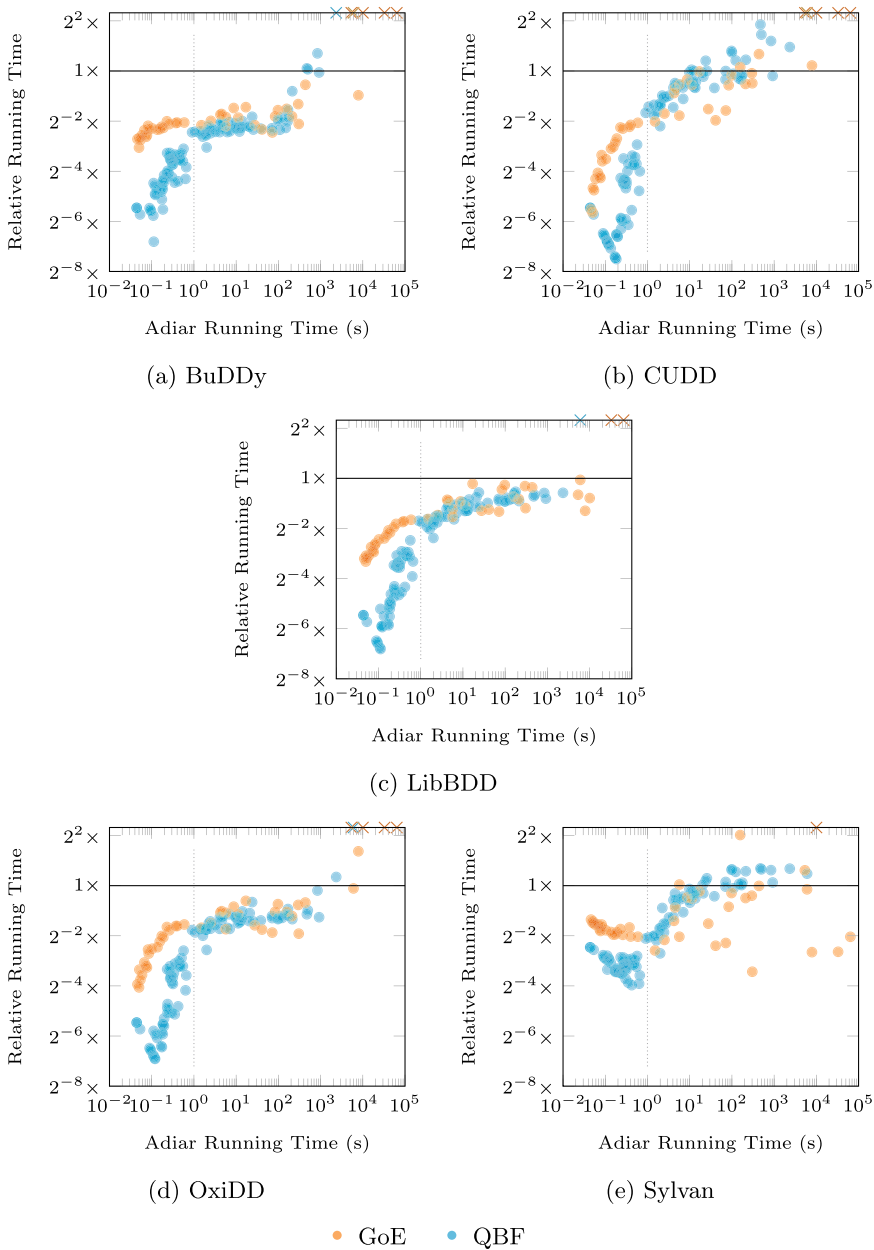


Fig. 14. Relative performance of depth-first implementations compared to Adiar with nested sweeping. Time-/Memouts are marked as crosses.

Adiar is shown in Fig. 14. Out of the 147 instances, 140 are solved by all depth-first BDD packages, i.e. the remaining 7 instances have at least one BDD package running out of memory (MO) or time (TO). Adiar solves all of them. Running out of time is most likely due to repeated need for garbage collection, which essentially is equivalent to an MO. Yet for fairness, Table 2 shows the total time for these 140 commonly solved instances. The average ratio, on the other hand, pertains to all instances solved by the respective BDD package.

The relative running time of BuDDy in Fig. 14a and OxiDD in Fig. 14d shows that Adiar’s performance can be divided into three categories: the *small* instances that takes Adiar less than 1 s to solve but is out of its (current) scope, the *moderate* instances where Adiar needs between 1 and 10^3 s to solve and it is up to a constant factor of 4 slower than other BDD packages, and the *large* instances beyond 10^3 s where other BDD packages slow down compared to Adiar due to limited internal memory and repeated garbage collection. While the distinction is not as clear for CUDD in Fig. 14b, LibBDD in Fig. 14c, and Sylvan in Fig. 14e, they also follow the same trend. This relative performance is similar to our previous results in [48–51]. That is, nested sweeping allows Adiar to compute quantifications at no additional cost to our previous work.

As shown in Table 2, Adiar solves the 40 common GoE instances in 2.7 h and the 100 common QBF instances in 1.25 h. This makes it as fast as CUDD for both sets of benchmarks and faster than OxiDD on GoE and Sylvan on QBF. BuDDy is faster for both benchmarks but its use of 32 bits indices limits the maximum BDD size it supports and hence the number of instances it can solve. As shown in Fig. 14c, LibBDD is the only BDD package that is consistently faster than Adiar (ignoring its three MOs). Most likely, this is due to its lack of a shared unique node table. This improves its cache locality and removes the need for expensive garbage collections. Hence, LibBDD can either fit its BDD computations into the internal memory (and it is faster than Adiar) or it aborts.

As is evident from Fig. 14e, Sylvan is comparatively good at some of the larger GoE instances, thereby beating all other BDD packages in the total time

Table 2. Total time needed by Adiar and conventional depth-first implementations to solve the 140 commonly solved instances. The average (geometric mean) covers all instances that were commonly solved by all BDD packages and where Adiar needed at least 1 s to solve. Ratios smaller than 1.00 means Adiar is slower.

	Time		# Solved		Avg. Ratio (1+s)	
	• GoE	• QBF	• GoE	• QBF	• GoE	• QBF
Adiar	9655.7 s	4499.4 s	45	102	–	–
BuDDy	4725.5 s	3793.1 s	40	100	0.30	0.25
CUDD	10892.8 s	4591.9 s	40	101	0.61	0.75
LibBDD	4365.7 s	2687.3 s	43	101	0.54	0.45
OxiDD	21223.9 s	2379.6 s	41	101	0.48	0.39
Sylvan	2925.4 s	5841.4 s	44	102	0.46	0.70

to solve its subset of the GoE benchmarks. Furthermore, it is the only other BDD package able to solve all QBF instances within the given time limit. This is in parts thanks to its small memory footprint per BDD node [20]. Yet, Sylvan requires a total of 5.0 h to solve all 102 QBF instances whereas Adiar only needed 3.6 h, making Sylvan a factor of 1.4 times slower than Adiar. If Sylvan was given access to more than a single thread, then it would, of course, be expected to be at least as fast if not much faster than Adiar [20].

Considering the 140 commonly solved instances are, by definition, not the main focus of the algorithms in Adiar and that the experiments have been designed in favour of the conventional BDD packages, it is not surprising that Adiar does not greatly outshine the other BDD packages. Even so, it is faster than some implementations in some cases – despite storing BDDs on disk. Most importantly, it solves more instances than any other BDD package, witnessed by the timeouts in Fig. 14.

5 Related Work

Many other implementations of BDDs also support quantification of multiple variables [9, 20, 25, 31, 43, 52]. All these are based on a nested (inner) operation being accumulated in an (outer) traversal of the input; the nested sweeping framework achieves the same within the time-forward processing paradigm [4, 17].

The CAL [43] BDD package (based on [7, 40]) is to the best of our knowledge the only implementation of BDDs also designed to compute on BDDs whose size exceed main memory. To do so, it uses breadth-first algorithms that are resolved level by level. For each level it still follows the conventional approach: a unique node table is used to manage BDD nodes while a polynomial running time is guaranteed by use of a memoisation table. These per-level hash tables, both in theory and in practice, put an upper bound on the maximum BDD width that CAL can support with a certain amount of internal memory [6].

To skip redundant computations, its quantification operation also required additional ideas particular to the design of CAL. By the nature of nested sweeping, our proposed algorithm is, unlike CAL, not able to skip redundant computations. Furthermore, the lack of a unique node table in Adiar requires our algorithms to retrace and copy the subtrees that are unchanged. Even so, as evident in Sect. 4, Adiar with nested sweeping outperforms CAL by up to several orders of magnitude. Moreover, the I/O-efficient approach in [6, 49], and by extension the ones in this work, are, unlike CAL, I/O efficient despite a BDD's level is wider than main memory.

6 Conclusions and Future Work

Each sweep in [49] is independent of the others. Using only this approach, one can only quantify a single variable a time but not multiple at once. In this work, we enable multi-variable quantification with the *nested sweeping* framework. Here,

multiple sweeps work together: each sweep forwards information within priority queues to itself, its parent, or its child in a recursion stack.

In practice, nested sweeping has improved the total time that Adiar needs to solve our quantification benchmarks by 21%. On average, it improves the running time of each instance by a factor of 1.7. This allows us to extend our previous results in [48–51] to Adiar’s quantification operations: ignoring small instances, Adiar is at most 4 times slower than conventional depth-first implementations. Adiar even outperforms depth-first implementations as they get closer to the limits of internal memory. As Adiar’s nested sweeping algorithms are implemented on-top of the I/O-efficient data structures that were also used in [48–51], its performance is unaffected by a limited internal memory [49]. For example, whereas CUDD [52] could solve 141 out of our 147 benchmark instances in 5.6 h, Adiar needed only 4.6 h to do the same and it could also solve the remaining 6 instances. Adiar is also faster, often by several orders of magnitude, than the only other external memory BDD package, CAL [43].

The nested sweeping framework has already been generalised to pave the way for the implementation of other multi-recursive BDD operations. We have used this to implement the relational product [47]; doing so requires several additional optimisations for its variable relabelling and its combined **and-exists** [47]. In [47], we also provide an evaluation on a large collection of symbolic model checking experiments. We also hope to use nested sweeping for functional composition and as the foundation for novel I/O-efficient variable reordering procedures [45]. Finally, nested sweeping opens up the possibility to create an I/O-efficient implementation of other types of decision diagrams. For example, both Quantum Multiple-valued Decision Diagrams [36] and Polynomial Boolean Rings [11] require nested sweeps to implement their multiplication operations.

Acknowledgements. Thanks to the Centre for Scientific Computing, Aarhus, for running our benchmarks on the Grendel cluster and thanks to Marijn Heule and Randal E. Bryant for suggesting the Garden of Eden problem and their ideas on how to encode it.

Data Availability Statement. The source code for all our benchmarks is available on the following GitHub repository at commit 51b1375: github.com/ssoelvsten/bdd-benchmark The source code is also available at [doi:10.5281/zenodo.4718224](https://doi.org/10.5281/zenodo.4718224) on Zenodo. The raw data and its analysis is available at [doi:10.5281/zenodo.17054026](https://doi.org/10.5281/zenodo.17054026).

References

1. Aggarwal, A., Vitter, J.S.: The input/output complexity of sorting and related problems. *Commun. ACM* **31**(9), 1116–1127 (1988). <https://doi.org/10.1145/48529.48535>
2. Akers, S.B.: Binary decision diagrams. *IEEE Trans. Comput.* **C-27**(6), 509–516 (1978). <https://doi.org/10.1109/TC.1978.1675141>
3. Amparore, E.G., Donatelli, S., Gallà, F.: starMC: an automata based CTL* model checker. *PeerJ Comput. Sci.* **8**, e823 (2022)

4. Arge, L.: The buffer tree: a new technique for optimal I/O-algorithms. In: Akl, S.G., Dehne, F., Sack, J.-R., Santoro, N. (eds.) WADS 1995. LNCS, vol. 955, pp. 334–345. Springer, Heidelberg (1995). https://doi.org/10.1007/3-540-60220-8_74
5. Arge, L.: The I/O-complexity of ordered binary-decision diagram manipulation. In: Proceedings of International Symposium on Algorithms and Computations, ISAAC’95, pp. 82–91 (1995)
6. Arge, L.: The I/O-complexity of ordered binary-decision diagram manipulation. In: Efficient External-Memory Data Structures and Applications (PhD Thesis), pp. 123–145. Aarhus Universitet, Datalogisk Institut (1996)
7. Ashar, P., Cheong, M.: Efficient breadth-first manipulation of binary decision diagrams. In: IEEE/ACM International Conference on Computer-Aided Design (ICCAD), pp. 622–627. IEEE Computer Society Press (1994). <https://doi.org/10.1109/ICCAD.1994.629886>
8. Beer, R.D.: Cultivating the garden of Eden. arXiv (2022). <https://doi.org/10.48550/arXiv.2210.07837>
9. Beneš, N., Brim, L., Kadlec, J., Pastva, S., Šafránek, D.: AEON: attractor bifurcation analysis of parametrised Boolean networks. In: Lahiri, S.K., Wang, C. (eds.) CAV 2020. LNCS, vol. 12224, pp. 569–581. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-53288-8_28
10. Brace, K.S., Rudell, R.L., Bryant, R.E.: Efficient implementation of a BDD package. In: 27th Design Automation Conference (DAC), pp. 40–45. Association for Computing Machinery (1990). <https://doi.org/10.1109/DAC.1990.114826>
11. Brickenstein, M., Dreyer, A.: PolyBoRi: a framework for Gröbner-basis computations with Boolean polynomials. *J. Symb. Comput.* **44**(9), 1326–1345 (2009). <https://doi.org/10.1016/j.jsc.2008.02.017>
12. Bryant, R.E.: Graph-based algorithms for boolean function manipulation. *IEEE Trans. Comput.* **C-35**(8), 677–691 (1986)
13. Bryant, R.E., Biere, A., Heule, M.J.H.: Clausal proofs for pseudo-boolean reasoning. In: TACAS 2022. LNCS, vol. 13243, pp. 443–461. Springer, Cham (2022). https://doi.org/10.1007/978-3-030-99524-9_25
14. Bryant, R.E., Heule, M.J.H.: Dual proof generation for quantified boolean formulas with a BDD-based solver. In: Platzer, A., Sutcliffe, G. (eds.) CADE 2021. LNCS (LNAI), vol. 12699, pp. 433–449. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-79876-5_25
15. Bryant, R.E., Heule, M.J.H.: Generating extended resolution proofs with a BDD-based SAT solver. In: TACAS 2021. LNCS, vol. 12651, pp. 76–93. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-72016-2_5
16. Chen, J., Revels, J.: Robust benchmarking in noisy environments. arXiv (2016). <https://arxiv.org/abs/1608.04295>
17. Chiang, Y.J., Goodrich, M.T., Grove, E.F., Tamassia, R., Vengroff, D.E., Vitter, J.S.: External-memory graph algorithms. In: Proceedings of the Sixth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA ’95, pp. 139–149. Society for Industrial and Applied Mathematics (1995)
18. Ciardo, G., Miner, A.S., Wan, M.: Advanced features in SMART: the stochastic model checking analyzer for reliability and timing. *SIGMETRICS Perf. Eval. Rev.* **36**(4), 58–63 (2009)
19. Cimatti, A., Clarke, E., Giunchiglia, F., Roveri, M.: NuSMV: a new symbolic model checker. *Int. J. Softw. Tools Technol. Transfer* **2**, 410–425 (2000). <https://doi.org/10.1007/s100090050046>

20. van Dijk, T., van de Pol, J.: Sylvan: multi-core framework for decision diagrams. *Int. J. Softw. Tools Technol. Transfer* 1–22 (2016). <https://doi.org/10.1007/s10009-016-0433-2>
21. Fried, D., Tabajara, L.M., Vardi, M.Y.: BDD-based boolean functional synthesis. In: Chaudhuri, S., Farzan, A. (eds.) *CAV 2016*. LNCS, vol. 9780, pp. 402–421. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-41540-6_22
22. Gammie, P., van der Meyden, R.: MCK: model checking the logic of knowledge. In: Alur, R., Peled, D.A. (eds.) *CAV 2004*. LNCS, vol. 3114, pp. 479–483. Springer, Heidelberg (2004). https://doi.org/10.1007/978-3-540-27813-9_41
23. Gardner, M.: The fantastic combinations of John Conway’s new solitaire game “life”. *Sci. Am.* **223**(4), 120–123 (1970). <https://doi.org/10.1038/scientificamerican1070-120>
24. He, L., Liu, G.: Petri net based symbolic model checking for computation tree logic of knowledge. *arXiv* (2020). <https://arxiv.org/abs/2012.10126>
25. Husung, N., Dubslaff, C., Hermanns, H., Köhl, M.A.: OxiDD: a safe, concurrent, modular, and performant decision diagram framework in Rust. In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS’24)*. LNCS, vol. 14572. Springer, Heidelberg (2024). https://doi.org/10.1007/978-3-031-57256-2_13
26. Kant, G., Laarman, A., Meijer, J., van de Pol, J., Blom, S., van Dijk, T.: LTSmin: high-performance language-independent model checking. In: Baier, C., Tinelli, C. (eds.) *TACAS 2015*. LNCS, vol. 9035, pp. 692–707. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-46681-0_61
27. Karplus, K.: Representing boolean functions with if-then-else DAGs. Technical report. University of California at Santa Cruz, USA (1988)
28. Klarlund, N., Rauhe, T.: BDD algorithms and cache misses. In: *BRICS Report Series*, vol. 26 (1996). <https://doi.org/10.7146/brics.v3i26.20007>
29. Lee, C.Y.: Representation of switching circuits by binary-decision programs. *Bell Syst. Techn. J.* **38**(4), 985–999 (1959). <https://doi.org/10.1002/j.1538-7305.1959.tb01585.x>
30. Lin, Y., Tabajara, L.M., Vardi, M.Y.: ZDD boolean synthesis. In: *TACAS 2022*. LNCS, vol. 13243, pp. 64–83. Springer, Cham (2022). https://doi.org/10.1007/978-3-030-99524-9_4
31. Lind-Nielsen, J.: BuDDy: a binary decision diagram package. Technical report, Department of Information Technology, Technical University of Denmark (1999)
32. Lomuscio, A., Qu, H., Raimondi, F.: MCMAS: an open-source model checker for the verification of multi-agent systems. *Int. J. Softw. Tools Technol. Transfer* **19**(1), 9–30 (2015). <https://doi.org/10.1007/s10009-015-0378-x>
33. Madsen, M., Van de Pol, J.: Polymorphic types and effects with Boolean unification. *Proc. ACM Program. Lang.* **4**(OOPSLA) (2020). <https://doi.org/10.1145/3428222>
34. Madsen, M., Van de Pol, J., Henriksen, T.: Fast and efficient boolean unification for Hindley-Milner-style type and effect systems. *Proc. ACM Program. Lang.* **7**(OOPSLA 2) (2023). <https://doi.org/10.1145/3622816>
35. Michaud, T., Colange, M.: Reactive synthesis from LTL specification with Spot. In: *7th Workshop on Synthesis, SYNT@ CAV*, vol. 5 (2018)
36. Miller, D., Thornton, M.: QMDD: a decision diagram structure for reversible and quantum circuits. In: *36th International Symposium on Multiple-Valued Logic*, pp. 30–36 (2006). <https://doi.org/10.1109/ISMVL.2006.35>
37. Minato, S.i., Ishihara, S.: Streaming BDD manipulation for large-scale combinatorial problems. In: *Design, Automation and Test in Europe Conference and Exhibition*, pp. 702–707 (2001). <https://doi.org/10.1109/DATE.2001.915104>

38. Minato, S.i., Ishiura, N., Yajima, S.: Shared binary decision diagram with attributed edges for efficient Boolean function manipulation. In: 27th Design Automation Conference, pp. 52–57. Association for Computing Machinery (1990). <https://doi.org/10.1145/123186.123225>
39. Moore, E.F.: Machine models of self-reproduction. In: Proceedings of Symposia in Applied Mathematics, vol. 14, pp. 17–33 (1962). <https://doi.org/10.1090/psapm/014>
40. Ochi, H., Yasuoka, K., Yajima, S.: Breadth-first manipulation of very large binary-decision diagrams. In: International Conference on Computer Aided Design (ICCAD), pp. 48–55. IEEE Computer Society Press (1993). <https://doi.org/10.1109/ICCAD.1993.580030>
41. Pastva, S., Henzinger, T.: Binary decision diagrams on modern hardware. In: Conference on Formal Methods in Computer-Aided Design, pp. 122–131 (2023)
42. Renkin, F., Schlehuber-Caissier, P., Duret-Lutz, A., Pommellet, A.: Dissecting ltl-synt. *Formal Methods Syst. Des.* **61**, 248–289 (2022). <https://doi.org/10.1007/s10703-022-00407-6>
43. Sanghavi, J.V., Ranjan, R.K., Brayton, R.K., Sangiovanni-Vincentelli, A.: High performance BDD package by exploiting memory hierarchy. In: 33rd Design Automation Conference (DAC), pp. 635–640. Association for Computing Machinery (1996). <https://doi.org/10.1145/240518.240638>
44. Shaik, I., Van de Pol, J.: Concise QBF encodings for games on a grid (extended version). arXiv (2023). <https://doi.org/10.48550/arXiv.2303.16949>
45. Sølvsten, S.C.: I/O-efficient Symbolic Model Checking. PhD thesis, Aarhus University (2025). https://soeg.kb.dk/permalink/45KBDK_KGL/1pioq0f/alma99126389524805763
46. Sølvsten, S.C., van de Pol, J.: Multi-variable quantification of BDDs in external memory using nested sweeping (extended paper). arXiv (2024). <https://arxiv.org/abs/2408.14216>
47. Sølvsten, S.C., van de Pol, J.: Symbolic model checking in external memory. arXiv (2025). <https://arxiv.org/abs/2505.11229>
48. Sølvsten, S.C., Van de Pol, J.: Adiar 1.1: Zero-suppressed decision diagrams in external memory. In: NASA Formal Methods Symposium. LNCS, vol. 13903, Springer, Heidelberg (2023). https://doi.org/10.1007/978-3-031-33170-1_28
49. Sølvsten, S.C., de Pol, J., Jakobsen, A.B., Thomasen, M.W.B.: Adiar binary decision diagrams in external memory. In: TACAS 2022. LNCS, vol. 13244, pp. 295–313. Springer, Cham (2022). https://doi.org/10.1007/978-3-030-99527-0_16
50. Sølvsten, S.C., Rysgaard, C.M., Van de Pol, J.: Random access on narrow decision diagrams in external memory. In: Model Checking Software, pp. 137–145. Springer, Heidelberg (2024). https://doi.org/10.1007/978-3-031-66149-5_7
51. Sølvsten, S.C., Van de Pol, J.: Predicting memory demands of BDD Operations using maximum graph cuts. In: André, É., Sun, J. (eds.) *Automated Technology for Verification and Analysis*. Lecture Notes in Computer Science, vol. 14216, pp. 72–92. Springer, Heidelberg (2023). https://doi.org/10.1007/978-3-031-45332-8_4
52. Somenzi, F.: CUDD: CU decision diagram package, 3.0. Technical report, University of Colorado at Boulder (2015)
53. Van Dijk, T., Van Abbema, F., Tomov, N.: Knor: reactive synthesis using oink. In: *Tools and Algorithms for the Construction and Analysis of Systems*. pp. 103–122. Springer, Heidelberg (2024). https://doi.org/10.1007/978-3-031-57246-3_7
54. QBF Gallery 2014: QCIR-G14: a non-prenex non-CNF format for quantified Boolean formulas (2014)